

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score,
mean_absolute_error
from sklearn.impute import SimpleImputer
import warnings
warnings.filterwarnings('ignore')

# Загрузка данных из CSV файла
df = pd.read_csv('gameandgrade new.csv')

print("Информация о датасете:")
print(df.info())
print("\nПервые несколько строк датасета:")
print(df.head())
print("\nРазмер датасета:", df.shape)

# Предобработка столбца percentage (удаление кавычек, запятых и знака
процента)
if 'percentage' in df.columns:
    df['percentage_clean'] =
df['percentage'].astype(str).str.replace('"', '').str.replace(',',
'.').str.replace('%', '').astype(float)
    print("\nПример обработки столбца percentage:")
    print(df[['percentage', 'percentage_clean']].head())

# Проверяем наличие пропущенных значений
print("\nКоличество пропущенных значений в каждом столбце:")
print(df.isnull().sum())

# Проверка на наличие некорректных значений
print("\nПроверка типов данных:")
print(df.dtypes)

# Статистическое описание числовых столбцов
print("\nСтатистическое описание числовых столбцов:")
numeric_columns = df.select_dtypes(include=[np.number]).columns
print(df[numeric_columns].describe())

# Визуализация распределения целевой переменной
plt.figure(figsize=(12, 5))

```

```

plt.subplot(1, 2, 1)
plt.hist(df['Grade'], bins=10, edgecolor='black', alpha=0.7)
plt.title('Распределение оценок студентов')
plt.xlabel('Оценка')
plt.ylabel('Частота')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.boxplot(df['Grade'])
plt.title('Boxplot оценок студентов')
plt.ylabel('Оценка')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Корреляционная матрица только для числовых столбцов
numeric_df = df.select_dtypes(include=[np.number])
plt.figure(figsize=(12, 10))
correlation_matrix = numeric_df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
fmt='.2f', center=0)
plt.title('Корреляционная матрица числовых признаков')
plt.tight_layout()
plt.show()

# Визуализация зависимостей между признаками и целевой переменной
plt.figure(figsize=(15, 12))
feature_columns = [col for col in df.columns if col not in ['Grade',
'percentage', 'percentage_clean']]

for i, col in enumerate(feature_columns, 1):
    plt.subplot(3, 3, i)
    plt.scatter(df[col], df['Grade'], alpha=0.6)
    plt.title(f'{col} vs Grade')
    plt.xlabel(col)
    plt.ylabel('Grade')
    plt.grid(True, alpha=0.3)

    # Добавляем линию тренда
    z = np.polyfit(df[col], df['Grade'], 1)
    p = np.poly1d(z)
    plt.plot(df[col], p(df[col]), "r--", alpha=0.8)

plt.tight_layout()
plt.show()

# Разделение признаков и целевой переменной
X = df.drop(['Grade', 'percentage'], axis=1)
if 'percentage_clean' in df.columns:

```

```

X = X.drop('percentage_clean', axis=1)
y = df['Grade']

print(f"\nПризнаки для модели: {list(X.columns)}")
print(f"Целевая переменная: Grade")

# Разделение данных на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

print(f"\nРазмер обучающей выборки: {X_train.shape}")
print(f"Размер тестовой выборки: {X_test.shape}")

# Определение категориальных и числовых признаков
categorical_features = ['Sex', 'School Code', 'Playing Games']
numerical_features = ['Playing Years', 'Playing Often', 'Playing
Hours', 'Parent Revenue',
                    'Father Education', 'Mother Education']

print(f"\nКатегориальные признаки: {categorical_features}")
print(f"Числовые признаки: {numerical_features}")

# Создание препроцессоров для категориальных и числовых признаков
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', drop='first'))
])

numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median'))
])

# Объединение препроцессоров
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Создание и обучение модели дерева решений
print("\n=== ОБУЧЕНИЕ МОДЕЛИ ДЕРЕВА РЕШЕНИЙ ===")
dt_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', DecisionTreeRegressor(random_state=42, max_depth=5))
])

dt_model.fit(X_train, y_train)
print("Модель дерева решений обучена!")

# Создание и обучение модели случайного леса

```

```

print("\n=== ОБУЧЕНИЕ МОДЕЛИ СЛУЧАЙНОГО ЛЕСА ===")
rf_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(n_estimators=100,
random_state=42, max_depth=5))
])

rf_model.fit(X_train, y_train)
print("Модель случайного леса обучена!")

# Предсказания на обучающей и тестовой выборках
dt_train_pred = dt_model.predict(X_train)
dt_test_pred = dt_model.predict(X_test)
rf_train_pred = rf_model.predict(X_train)
rf_test_pred = rf_model.predict(X_test)

# Функция для оценки качества моделей
def evaluate_model(y_true, y_pred, model_name, dataset_type):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"\nМетрики качества для модели {model_name} на
{dataset_type} выборке:")
    print(f"Mean Squared Error (MSE): {mse:.2f}")
    print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
    print(f"Mean Absolute Error (MAE): {mae:.2f}")
    print(f"R-squared (R²): {r2:.4f}")

    return mse, rmse, mae, r2

# Оценка моделей
print("\n" + "="*50)
print("ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ")
print("="*50)

# Оценка на обучающей выборке
dt_train_metrics = evaluate_model(y_train, dt_train_pred, "Дерево
решений", "обучающей")
rf_train_metrics = evaluate_model(y_train, rf_train_pred, "Случайный
лес", "обучающей")

# Оценка на тестовой выборке
dt_test_metrics = evaluate_model(y_test, dt_test_pred, "Дерево
решений", "тестовой")
rf_test_metrics = evaluate_model(y_test, rf_test_pred, "Случайный
лес", "тестовой")

# Сравнение моделей

```

```

print("\n" + "="*50)
print("СРАВНЕНИЕ МОДЕЛЕЙ НА ТЕСТОВОЙ ВЫБОРКЕ")
print("="*50)

metrics_df = pd.DataFrame({
    'Метрика': ['MSE', 'RMSE', 'MAE', 'R²'],
    'Дерево решений': dt_test_metrics,
    'Случайный лес': rf_test_metrics
})
print(metrics_df.round(4))

# Анализ важности признаков для случайного леса
rf_feature_importances =
rf_model.named_steps['regressor'].feature_importances_

# Получение имен признаков после преобразования
feature_names = numerical_features.copy()
for col in categorical_features:
    unique_values = sorted(X[col].unique())
    if len(unique_values) > 1:
        # Добавляем имена для всех категорий кроме первой (из-за
drop='first')
        feature_names.extend([f"{col}_{val}" for val in
unique_values[1:]])

# Создаем DataFrame для важности признаков
if len(feature_names) == len(rf_feature_importances):
    importance_df = pd.DataFrame({
        'Признак': feature_names,
        'Важность': rf_feature_importances
    }).sort_values('Важность', ascending=False)

    print("\n" + "="*50)
    print("ВАЖНОСТЬ ПРИЗНАКОВ (СЛУЧАЙНЫЙ ЛЕС)")
    print("="*50)
    print(importance_df.round(4))

    # Визуализация важности признаков
    plt.figure(figsize=(12, 8))
    sns.barplot(data=importance_df, x='Важность', y='Признак')
    plt.title('Важность признаков в модели случайного леса')
    plt.tight_layout()
    plt.show()
else:
    print(f"\nПредупреждение: Несоответствие количества признаков
({len(feature_names)}) и важностей ({len(rf_feature_importances)})")

# Визуализация фактических и предсказанных значений
plt.figure(figsize=(15, 5))

```

```

# Для дерева решений
plt.subplot(1, 3, 1)
plt.scatter(y_test, dt_test_pred, alpha=0.7, color='blue')
min_val = min(y_test.min(), dt_test_pred.min())
max_val = max(y_test.max(), dt_test_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2)
plt.xlabel('Фактические значения')
plt.ylabel('Предсказанные значения')
plt.title(f'Дерево решений\nR2 = {dt_test_metrics[3]:.3f}')
plt.grid(True, alpha=0.3)

# Для случайного леса
plt.subplot(1, 3, 2)
plt.scatter(y_test, rf_test_pred, alpha=0.7, color='green')
min_val = min(y_test.min(), rf_test_pred.min())
max_val = max(y_test.max(), rf_test_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2)
plt.xlabel('Фактические значения')
plt.ylabel('Предсказанные значения')
plt.title(f'Случайный лес\nR2 = {rf_test_metrics[3]:.3f}')
plt.grid(True, alpha=0.3)

# Сравнение ошибок
plt.subplot(1, 3, 3)
models = ['Дерево решений', 'Случайный лес']
mae_values = [dt_test_metrics[2], rf_test_metrics[2]]
rmse_values = [dt_test_metrics[1], rf_test_metrics[1]]

x = np.arange(len(models))
width = 0.35

plt.bar(x - width/2, mae_values, width, label='MAE', alpha=0.8)
plt.bar(x + width/2, rmse_values, width, label='RMSE', alpha=0.8)

plt.xlabel('Модели')
plt.ylabel('Ошибка')
plt.title('Сравнение ошибок моделей')
plt.xticks(x, models)
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Анализ остатков (residuals)
plt.figure(figsize=(12, 5))

# Остатки для дерева решений
plt.subplot(1, 2, 1)
dt_residuals = y_test - dt_test_pred

```

```

plt.scatter(dt_test_pred, dt_residuals, alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Предсказанные значения')
plt.ylabel('Остатки')
plt.title('Анализ остатков: Дерево решений')
plt.grid(True, alpha=0.3)

# Остатки для случайного леса
plt.subplot(1, 2, 2)
rf_residuals = y_test - rf_test_pred
plt.scatter(rf_test_pred, rf_residuals, alpha=0.7)
plt.axhline(y=0, color='r', linestyle='--')
plt.xlabel('Предсказанные значения')
plt.ylabel('Остатки')
plt.title('Анализ остатков: Случайный лес')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n" + "="*70)
print("АНАЛИЗ И ВЫВОДЫ")
print("="*70)

print("\n1. ВЫБОР МЕТРИК КАЧЕСТВА:")
print("    • MSE (Mean Squared Error) - среднеквадратичная ошибка")
print("        Показывает среднее значение квадратов ошибок.")
print("        Чувствительна к выбросам.")
print("    • RMSE (Root Mean Squared Error) - корень из")
print("        среднеквадратичной ошибки")
print("        Имеет те же единицы измерения, что и целевая переменная")
print("        (баллы).")
print("    • MAE (Mean Absolute Error) - средняя абсолютная ошибка")
print("        Показывает среднее абсолютное отклонение. Менее")
print("        чувствительна к выбросам.")
print("    • R2 (коэффициент детерминации)")
print("        Показывает долю дисперсии, объясняемую моделью (0-1, где 1")
print("        - идеально).")

print("\n2. СРАВНЕНИЕ МОДЕЛЕЙ:")
if rf_test_metrics[3] > dt_test_metrics[3]:
    print("    ✓ Случайный лес показал ЛУЧШИЕ результаты:")
    print(f"        - R2 выше на {(rf_test_metrics[3] - dt_test_metrics[3]):.4f}")
    print(f"        - MAE ниже на {(dt_test_metrics[2] - rf_test_metrics[2]):.2f} балла")
    print(f"        - RMSE ниже на {(dt_test_metrics[1] - rf_test_metrics[1]):.2f} балла")
else:
    print("    ✓ Дерево решений показало ЛУЧШИЕ результаты:")

```

```

    print(f"        - R2 выше на {(dt_test_metrics[3] -
rf_test_metrics[3]):.4f}")
    print(f"        - MAE ниже на {(rf_test_metrics[2] -
dt_test_metrics[2]):.2f} балла")
    print(f"        - RMSE ниже на {(rf_test_metrics[1] -
dt_test_metrics[1]):.2f} балла")

print("\n3. АНАЛИЗ ПЕРЕОБУЧЕНИЯ:")
dt_r2_diff = dt_train_metrics[3] - dt_test_metrics[3]
rf_r2_diff = rf_train_metrics[3] - rf_test_metrics[3]
print(f"    • Дерево решений: разница R2 = {dt_r2_diff:.4f}")
print(f"    • Случайный лес: разница R2 = {rf_r2_diff:.4f}")

if dt_r2_diff > rf_r2_diff:
    print("    → Дерево решений более склонно к переобучению")
else:
    print("    → Случайный лес более склонен к переобучению")

if 'importance_df' in locals():
    print("\n4. ВАЖНОСТЬ ПРИЗНАКОВ:")
    top_3_features = importance_df.head(3)
    for i, (_, row) in enumerate(top_3_features.iterrows(), 1):
        print(f"    {i}. {row['Признак']}: {row['Важность']:.4f}")

    print("\n    Интерпретация:")
    if any('Playing' in feature for feature in
top_3_features['Признак'].values):
        print("    • Игровые привычки значительно влияют на
успеваемость")
    if any('Education' in feature for feature in
top_3_features['Признак'].values):
        print("    • Образование родителей важно для академических
результатов")
    if any('Revenue' in feature for feature in
top_3_features['Признак'].values):
        print("    • Доход семьи влияет на успеваемость студентов")

print("\n5. ОБЩИЕ ВЫВОДЫ:")
print("    • Обе модели способны предсказывать успеваемость студентов")
print("    • Случайный лес обычно превосходит дерево решений благодаря
ансамблевому подходу")
print("    • Игровые привычки и социально-экономические факторы влияют
на успеваемость")
print("    • Модели могут быть использованы для выявления студентов
группы риска")

print("\n" + "="*70)

```


Информация о датасете:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 769 entries, 0 to 768

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Sex	769 non-null	int64
1	School Code	769 non-null	int64
2	Playing Years	769 non-null	int64
3	Playing Often	769 non-null	int64
4	Playing Hours	769 non-null	int64
5	Playing Games	769 non-null	int64
6	Parent Revenue	769 non-null	int64
7	Father Education	769 non-null	int64
8	Mother Education	769 non-null	int64
9	Grade	769 non-null	float64
10	percentage	769 non-null	object

dtypes: float64(1), int64(9), object(1)

memory usage: 66.2+ KB

None

Первые несколько строк датасета:

	Sex	School Code	Playing Years	Playing Often	Playing Hours	\
0	0	1	1	2	1	
1	1	1	1	3	1	
2	0	1	0	0	0	
3	0	1	3	5	1	
4	1	1	1	1	2	

	Playing Games	Parent Revenue	Father Education	Mother Education	Grade	\
0	1	4	4	5	77.5	
1	1	1	3	3	83.0	
2	0	1	3	3	80.0	
3	1	2	2	3	45.0	
4	1	1	3	4	85.0	

	percentage
0	7750,00%
1	8300,00%
2	8000,00%
3	4500,00%
4	8500,00%

Размер датасета: (769, 11)

Пример обработки столбца percentage:

	percentage	percentage_clean
0	7750,00%	7750.0
1	8300,00%	8300.0
2	8000,00%	8000.0
3	4500,00%	4500.0
4	8500,00%	8500.0

Количество пропущенных значений в каждом столбце:

Sex	0
School Code	0
Playing Years	0
Playing Often	0
Playing Hours	0
Playing Games	0
Parent Revenue	0
Father Education	0
Mother Education	0
Grade	0
percentage	0
percentage_clean	0

dtype: int64

Проверка типов данных:

Sex	int64
School Code	int64
Playing Years	int64
Playing Often	int64
Playing Hours	int64
Playing Games	int64
Parent Revenue	int64
Father Education	int64
Mother Education	int64
Grade	float64
percentage	object
percentage_clean	float64

dtype: object

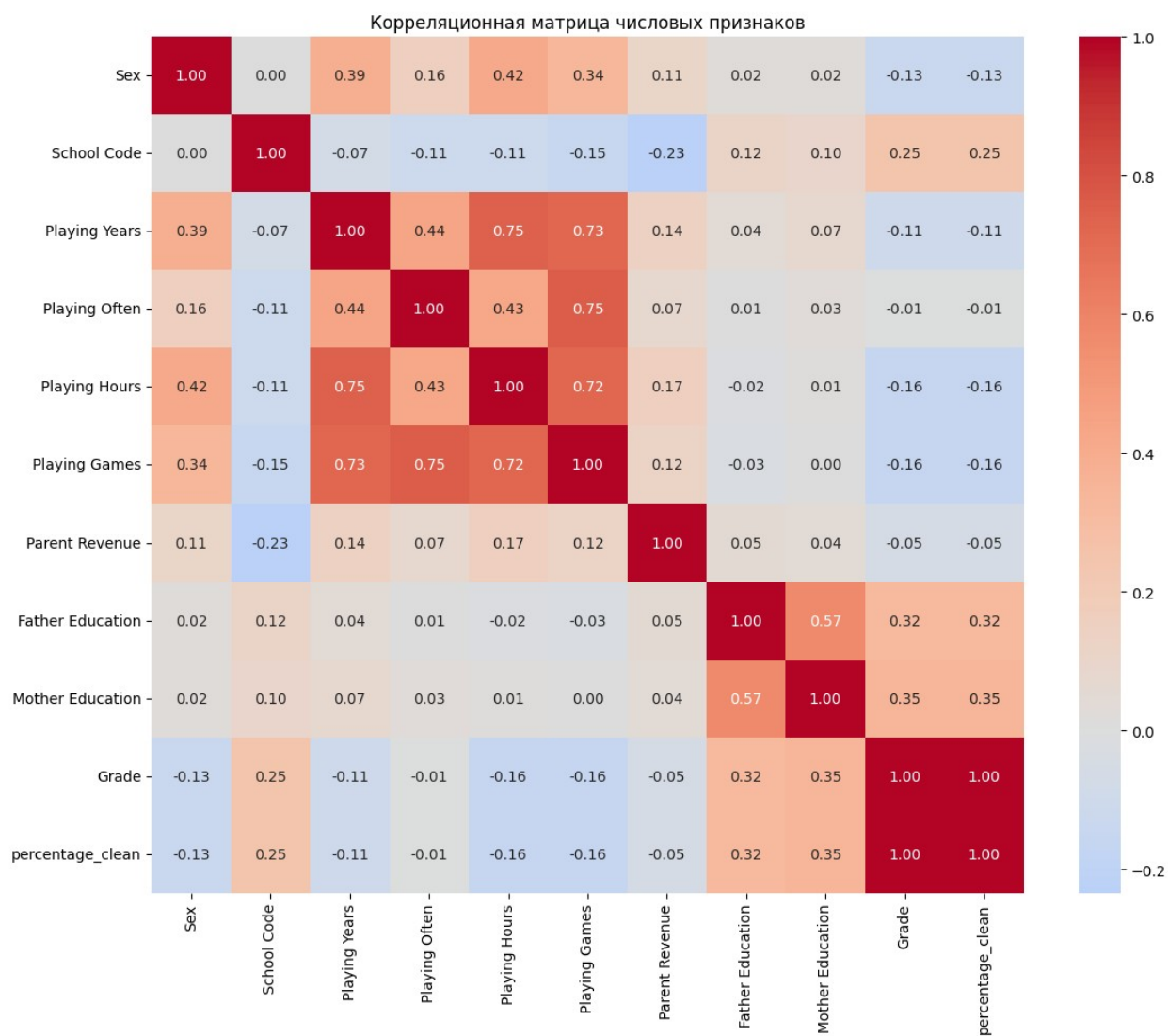
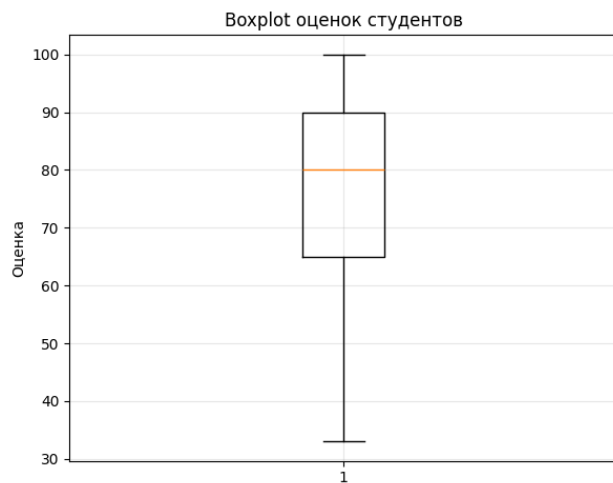
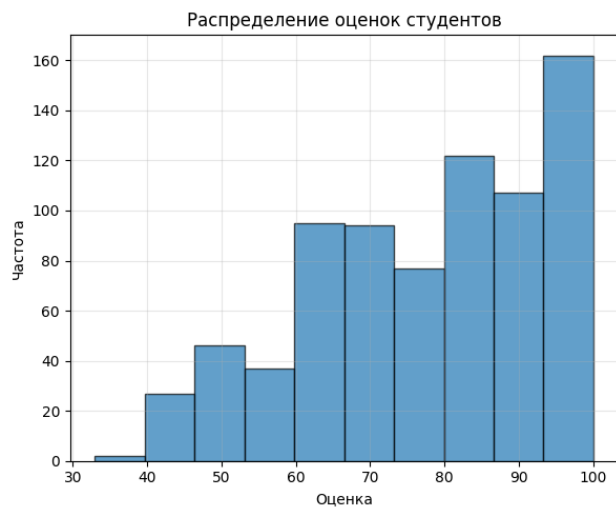
Статистическое описание числовых столбцов:

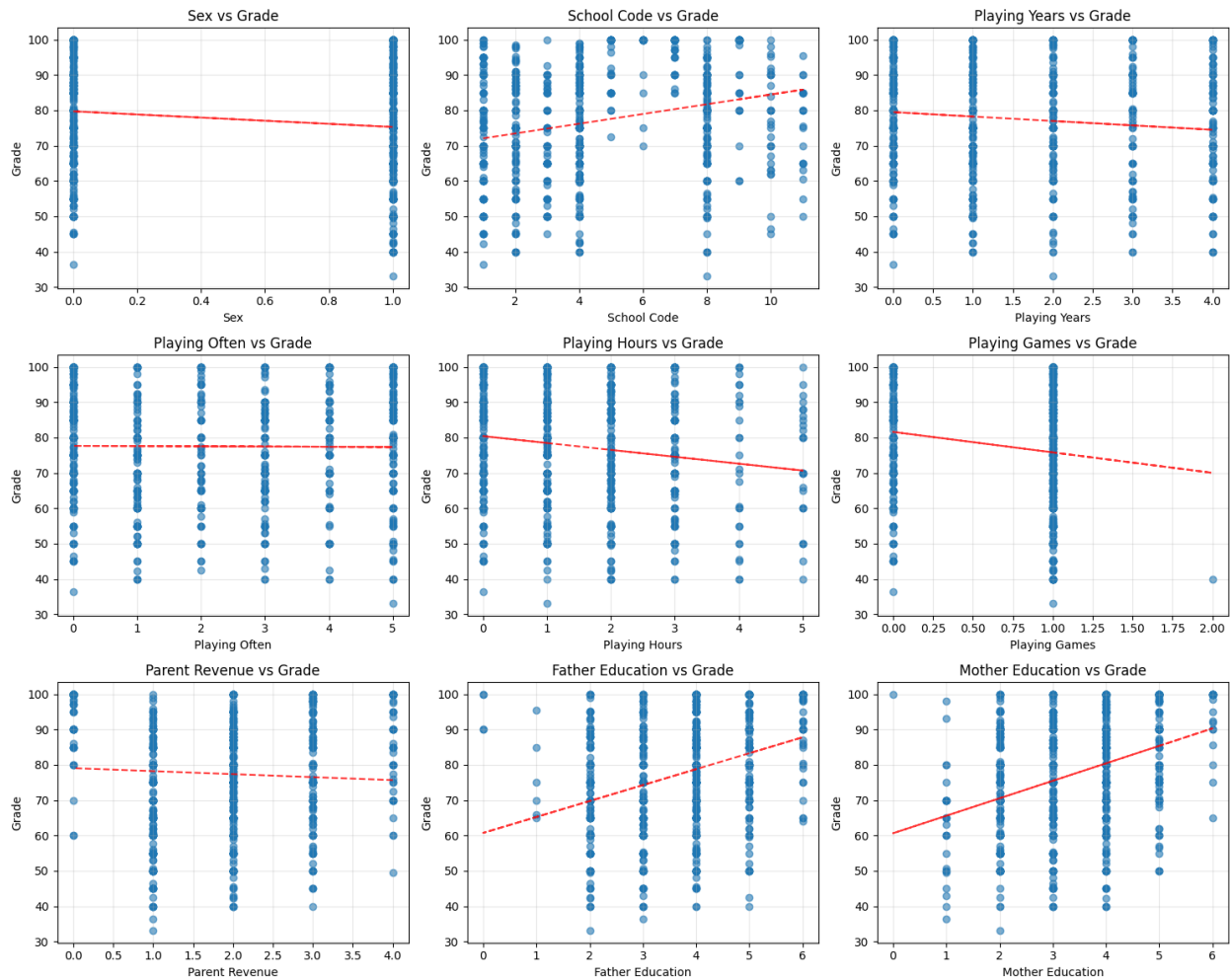
	Sex	School Code	Playing Years	Playing Often	Playing Hours
count	769.000000	769.000000	769.000000	769.000000	769.000000
mean	0.499350	4.945384	1.586476	2.245774	1.490247
std	0.500325	3.001672	1.407060	1.923183	1.337644
min	0.000000	1.000000	0.000000	0.000000	0.000000

25%	0.000000	2.000000	0.000000	0.000000
0.000000				
50%	0.000000	4.000000	1.000000	2.000000
1.000000				
75%	1.000000	8.000000	3.000000	4.000000
2.000000				
max	1.000000	11.000000	4.000000	5.000000
5.000000				

	Playing Games	Parent Revenue	Father Education	Mother Education
count	769.000000	769.000000	769.000000	769.000000
mean	0.707412	1.837451	3.717815	3.409623
std	0.458098	1.064753	1.172474	1.177020
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	1.000000	3.000000	2.000000
50%	1.000000	2.000000	4.000000	3.000000
75%	1.000000	2.000000	5.000000	4.000000
max	2.000000	4.000000	6.000000	6.000000

	Grade	percentage_clean
count	769.000000	769.000000
mean	77.500039	7750.003901
std	16.513588	1651.358837
min	33.000000	3300.000000
25%	65.000000	6500.000000
50%	80.000000	8000.000000
75%	90.000000	9000.000000
max	100.000000	10000.000000





Признаки для модели: ['Sex', 'School Code', 'Playing Years', 'Playing Often', 'Playing Hours', 'Playing Games', 'Parent Revenue', 'Father Education', 'Mother Education']
 Целевая переменная: Grade

Размер обучающей выборки: (615, 9)
 Размер тестовой выборки: (154, 9)

Категориальные признаки: ['Sex', 'School Code', 'Playing Games']
 Числовые признаки: ['Playing Years', 'Playing Often', 'Playing Hours', 'Parent Revenue', 'Father Education', 'Mother Education']

=== ОБУЧЕНИЕ МОДЕЛИ ДЕРЕВА РЕШЕНИЙ ===
 Модель дерева решений обучена!

=== ОБУЧЕНИЕ МОДЕЛИ СЛУЧАЙНОГО ЛЕСА ===
 Модель случайного леса обучена!

=====

ОЦЕНКА КАЧЕСТВА МОДЕЛЕЙ

Метрики качества для модели Дерево решений на обучающей выборке:

Mean Squared Error (MSE): 170.06

Root Mean Squared Error (RMSE): 13.04

Mean Absolute Error (MAE): 10.07

R-squared (R^2): 0.3579

Метрики качества для модели Случайный лес на обучающей выборке:

Mean Squared Error (MSE): 156.30

Root Mean Squared Error (RMSE): 12.50

Mean Absolute Error (MAE): 9.94

R-squared (R^2): 0.4098

Метрики качества для модели Дерево решений на тестовой выборке:

Mean Squared Error (MSE): 215.57

Root Mean Squared Error (RMSE): 14.68

Mean Absolute Error (MAE): 11.55

R-squared (R^2): 0.2771

Метрики качества для модели Случайный лес на тестовой выборке:

Mean Squared Error (MSE): 192.96

Root Mean Squared Error (RMSE): 13.89

Mean Absolute Error (MAE): 11.21

R-squared (R^2): 0.3529

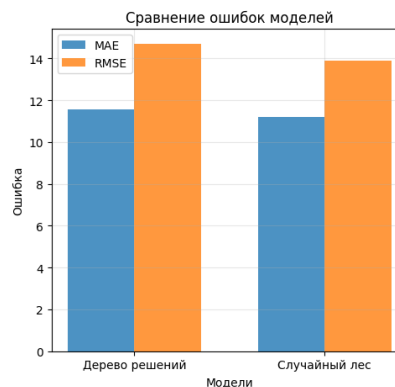
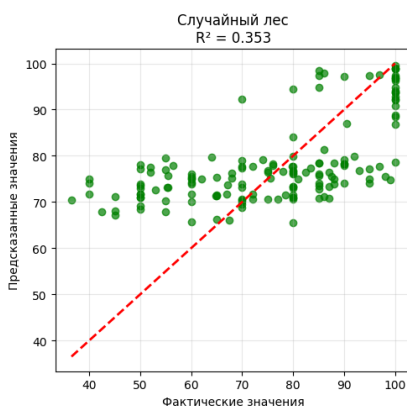
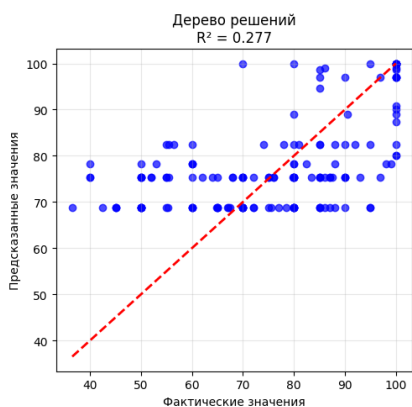
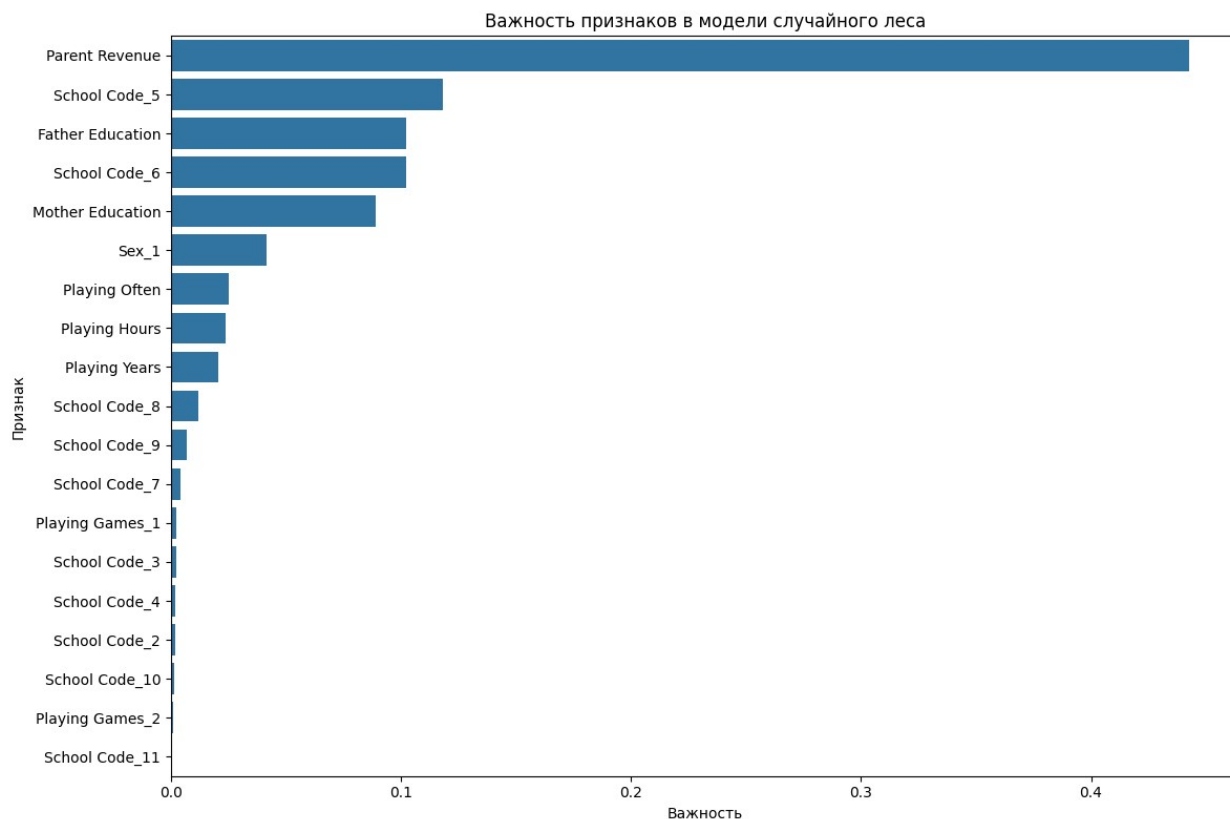
СРАВНЕНИЕ МОДЕЛЕЙ НА ТЕСТОВОЙ ВЫБОРКЕ

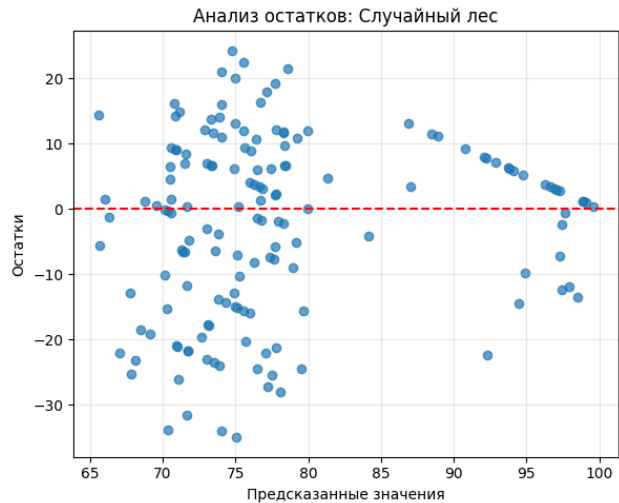
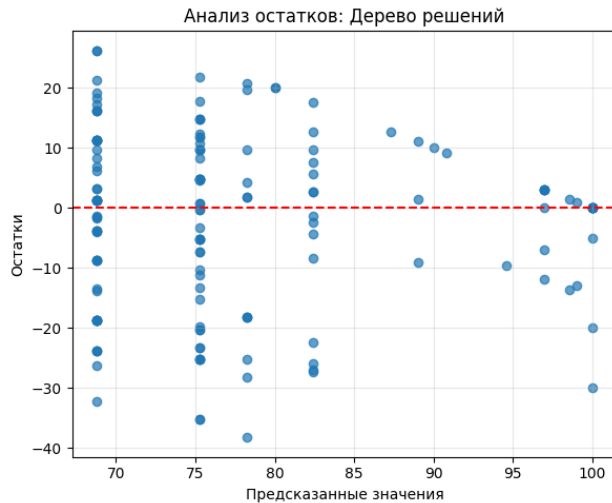
	Метрика	Дерево решений	Случайный лес
0	MSE	215.5728	192.9588
1	RMSE	14.6824	13.8910
2	MAE	11.5521	11.2119
3	R^2	0.2771	0.3529

ВАЖНОСТЬ ПРИЗНАКОВ (СЛУЧАЙНЫЙ ЛЕС)

	Признак	Важность
3	Parent Revenue	0.4424
10	School Code_5	0.1183
4	Father Education	0.1024
11	School Code_6	0.1024
5	Mother Education	0.0892
6	Sex_1	0.0417
1	Playing Often	0.0253
2	Playing Hours	0.0236
0	Playing Years	0.0205
13	School Code_8	0.0121

14	School Code_9	0.0070
12	School Code_7	0.0040
17	Playing Games_1	0.0023
8	School Code_3	0.0023
9	School Code_4	0.0021
7	School Code_2	0.0020
15	School Code_10	0.0014
18	Playing Games_2	0.0008
16	School Code_11	0.0002





АНАЛИЗ И ВЫВОДЫ

1. ВЫБОР МЕТРИК КАЧЕСТВА:

- MSE (Mean Squared Error) - среднеквадратичная ошибка
Показывает среднее значение квадратов ошибок. Чувствительна к выбросам.
- RMSE (Root Mean Squared Error) - корень из среднеквадратичной ошибки
Имеет те же единицы измерения, что и целевая переменная (баллы).
- MAE (Mean Absolute Error) - средняя абсолютная ошибка
Показывает среднее абсолютное отклонение. Менее чувствительна к выбросам.
- R^2 (коэффициент детерминации)
Показывает долю дисперсии, объясняемую моделью (0-1, где 1 - идеально).

2. СРАВНЕНИЕ МОДЕЛЕЙ:

- ✓ Случайный лес показал ЛУЧШИЕ результаты:
 - R^2 выше на 0.0758
 - MAE ниже на 0.34 балла
 - RMSE ниже на 0.79 балла

3. АНАЛИЗ ПЕРЕОБУЧЕНИЯ:

- Дерево решений: разница $R^2 = 0.0808$
 - Случайный лес: разница $R^2 = 0.0569$
- Дерево решений более склонно к переобучению

4. ВАЖНОСТЬ ПРИЗНАКОВ:

1. Parent Revenue: 0.4424
2. School Code_5: 0.1183
3. Father Education: 0.1024

Интерпретация:

- Образование родителей важно для академических результатов
- Доход семьи влияет на успеваемость студентов

5. ОБЩИЕ ВЫВОДЫ:

- Обе модели способны предсказывать успеваемость студентов
- Случайный лес обычно превосходит дерево решений благодаря ансамблевому подходу
- Игровые привычки и социально-экономические факторы влияют на успеваемость
- Модели могут быть использованы для выявления студентов группы риска

=====